

emulsim: A Python package for simulating complex emulsions

David Zwicker ¹

¹ Max Planck Institute for Dynamics and Self-Organization, Göttingen, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright,
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

Summary

Phase separation is an intrinsic property of mixtures that is widely observed in many scenarios, ranging from the simple demixing of oil and water to the condensation of biomolecules in cells ([Hyman et al., 2014](#)). In multicomponent mixtures, phase separation can lead to many coexisting phases, which is crucial in many fields. One key step to understand phase separation is to measure or predict the composition of the coexisting phases. To support such research, the flory package provides an easily accessible, performant, and extensible code that finds coexisting phases in multicomponent mixtures. The package expresses the free energy of the mixtures by several orthogonal aspects to cover a broad range of physical situations. In contrast to existing methods, the flory package implements a state-of-art method that is optimized for mixtures of many components. The package mainly focus on the mixtures with uniform and relatively simple free energies such as Flory-Huggins free energy and its generalizations, while more complicated ones can also be supported through extensions.

Statement of need

- Current approach: MD or other particle based simulations -> too complex
- Field-based approach, i.e., based on py-pde
- Custom code, which can be hard to develop

We have developed a general method in (?), but the code is not easily accessible and extensible.

Methods

The emulsim packages builds on a quite general simulation framework, where the state of the system is described by various *elements*. This state then evolves in times using various *actors*, which affect one or many elements. Building on this general principle, the package particularly provides elements that are useful to described emulsions and their surroundings. For instance, we provide the basic class SphericalDropletsElement, which describes a collection of droplets by their positions and sizes in one to three dimensions. We also provide a class MulticomponentDroplet, which additionally allows specifying the composition of droplets. Finally, we provide various elements to describe the surroundings of droplets, most notably the MeanfieldElement, which assumes a homogeneous composition, and ScalarFieldElement, where a scalar field captures the background composition. All element classes provide general methods for storing, reading, and plotting data to enable an interactive workflow based on jupyter notebooks. The state of the system is described by an instance of the State class, which may contain multiple different elements.

Autonomous actors

Coupling actors

39 Simulation approach

40 Examples

41 The following code illustrates the main functionality of the package by finding the two coexisting
42 phases of a symmetric Flory-Huggins binary mixture with $\chi = 4$:

```
from droplets import SphericalDroplet
from pde import ScalarField, UnitGrid

import sim

# set up state
grid = UnitGrid([32, 32], periodic=True)
background = sim.ScalarFieldElement.from_field(ScalarField(grid, 0.1))
droplet_data = [SphericalDroplet(grid.get_random_point(), 0.5) for _ in range(10)]
droplets = sim.SphericalDropletsElement.from_droplets(droplet_data)
state = sim.State({"background": background, "droplets": droplets})

# set up simulation
simulation = sim.Simulation(state)
simulation.add_actor("background", sim.DiffusionActor())
simulation.add_actor(("droplets", "background"), sim.SphericalDropletActor())

# run simulation
result = simulation.run(t_range=10)

result.plot()
```

43 Moreover, one can vary the type of interaction by initializing a different class or by modifying
44 the existing one, and one could similarly change the entropy, ensemble, and constraints.
45 Customized specialization of all four aspects can be easily implemented by deriving from the
46 provided base classes (*"Flory" Python Package — Flory Documentation, 2025*).

47 Acknowledgements

48 I thank all current and past members of the Zwicker group for stimulating discussions. We
49 gratefully acknowledge funding from the Max Planck Society and the European Union (ERC,
50 EmulSim, 101044662).

51 References

- 52 *"Flory" Python package — flory documentation.* (2025). <https://flory.readthedocs.io>
- 53 Hyman, A. A., Weber, C. A., & Jülicher, F. (2014). Liquid-Liquid Phase Separation in Biology.
54 *Annual Review of Cell and Developmental Biology*, 30(1), 39–58. [https://doi.org/10.1146/](https://doi.org/10.1146/annurev-cellbio-100913-013325)
55 [annurev-cellbio-100913-013325](https://doi.org/10.1146/annurev-cellbio-100913-013325)